Employee Management System Detailed Project Report

Abstract

The Employee Management System is a full-stack web application developed to automate employee administration within an organization. The application replaces manual record management with a secure centralized platform that allows administrators to manage employees, departments and users efficiently.

The project uses Spring Boot for backend services, React with TypeScript for the frontend, PostgreSQL as the database, JWT for authentication, Docker for containerization, Swagger for API documentation and Bootstrap for a responsive user interface.

The system demonstrates modern enterprise development practices including layered architecture, RESTful APIs, authentication, authorization, validation, pagination, file upload, reporting and dashboard analytics.

Chapter 1 – Introduction

Organizations require accurate employee records for payroll, department allocation and reporting.

Traditional manual systems are slow, error-prone and difficult to maintain.

This project provides a digital solution that securely stores employee information, supports searching, editing and reporting while maintaining data integrity. The system follows enterprise software engineering principles and is designed to be scalable, maintainable and easy to deploy.

Chapter 2 – Objectives

Primary objectives include:

1. Develop secure authentication using JWT.
2. Implement role-based authorization for ADMIN and USER.
3. Create complete CRUD operations for employee management.
4. Display dashboard statistics and charts.
5. Support employee photo upload.
6. Export employee data to PDF and Excel.
7. Provide REST APIs documented using Swagger.
8. Deploy using Docker and PostgreSQL.

Chapter 3 - Technology Stack

Backend Technologies:

- Java 17
- Spring Boot
- Spring Security
- Spring Data JPA
- Hibernate
- JWT Authentication
- Maven

Frontend Technologies:

- React
- TypeScript
- Bootstrap
- Axios
- React Router
- React Toastify
- Recharts

Database:

- PostgreSQL

DevOps:

- Docker• Docker Compose

Documentation:

- Swagger/OpenAPI

Spring Boot Actuator

Chapter 4 - System Architecture

The architecture follows a client-server model.

The React frontend communicates with Spring Boot REST APIs through HTTP requests.

Spring Security validates JWT tokens before processing secured requests.

Business logic is implemented in the Service layer.

Repositories communicate with PostgreSQL using Spring Data JPA.

Docker Compose orchestrates the application and database containers.

Chapter 5 - Functional Modules

Authentication Module:

Provides registration, login and JWT token generation.

Employee Module:

Supports create, read, update and delete operations.

Dashboard Module:

Displays total employees, departments, administrators and users together with department-wise pie charts.

Search Module:

Allows searching employees by name and department.

Export Module:

Generates Excel spreadsheets and PDF reports.

Administration Module:

Manages users, roles and secure access.

Chapter 6 - Database Design

The PostgreSQL database stores employee and authentication information.

Entities include User, Employee and Department.

Primary keys uniquely identify each record.

Foreign-key relationships maintain consistency between departments and employees. Spring Data JPA automatically maps Java entities to database tables.

Chapter 7 - Security

Security is implemented using Spring Security and JWT.
Passwords are encrypted using BCrypt.
Public endpoints are limited to authentication APIs.
All employee management APIs require authentication.
Role-based authorization prevents unauthorized access to administrative features.

Chapter 8 - API Documentation

Swagger provides interactive API documentation.
Major endpoints include:
POST /auth/login
POST /auth/register
GET /employees
POST /employees
PUT /employees/{id}
DELETE /employees/{id}
GET /dashboard
GET /dashboard/chart

Chapter 9 - Testing

The project was tested using Swagger, browser testing and JUnit.

Validation verified login, registration, CRUD operations, file upload, dashboard loading, search functionality, pagination, PDF export, Excel export and Docker deployment. Negative test cases confirmed invalid credentials and validation failures.

Chapter 10 - Docker Deployment

A multi-stage Dockerfile builds the Spring Boot application using Maven and packages the executable JAR into a lightweight runtime image.

Docker Compose starts PostgreSQL and the Spring Boot application together. Environment variables configure database connectivity.

Chapter 11 - Results

The completed application successfully manages employee records through an intuitive interface. Users can securely log in, maintain employee records, upload photographs, generate reports, visualize department statistics and export information. Docker deployment ensures consistent execution across environments.

Chapter 12 - Future Enhancements

Future improvements include:

- Email notifications
- Password reset
- Audit logs
- Redis caching
- Kubernetes deployment
- CI/CD pipelines
- Cloud monitoring
- Mobile application
- Attendance and payroll modules
- AI-powered analytics

Conclusion

The Employee Management System successfully demonstrates enterprise-level full-stack development.

The project integrates frontend, backend, database, authentication, documentation and deployment into a single maintainable solution. It highlights practical skills in Java, Spring Boot, React, TypeScript, PostgreSQL and Docker, making it suitable for academic submission and professional portfolios.